

AI CRICKET COACH

PROJECT REPORT PHASE II

Submitted by

Abiya Mariyam Alex	PRC22CSOT003
M Yadhu Krishna	PRC22CSOT016
Nandana Biju	PRC22CSOT017
Sania Sajan John	PRC22CSOT022

to

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of B.Tech Degree in
Internet of Things and Cyber Security



PROVIDENCE
COLLEGE OF ENGINEERING

Department of Internet of Things and Cyber Security

Providence College of Engineering, Chengannur

March 2026

PROVIDENCE COLLEGE OF ENGINEERING, CHENGANNUR**DEPARTMENT OF INTERNET OF THINGS AND CYBER SECURITY**

(Affiliated to APJ Abdul Kalam Technological University, Kerala)

**PROVIDENCE**
COLLEGE OF ENGINEERING**CERTIFICATE**

This is to certify that the Project Phase II report entitled **AI Cricket Coach** submitted by the following students in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Internet of Things and Cyber Security under APJ Abdul Kalam Technological University, is a bonafide record of the project work carried out during the academic year 2025–2026. The project has been presented and approved as per the prescribed guidelines for the course CGD 416 Project Phase II of the curriculum.

Abiya Mariyam Alex	PRC22CSOT003
M Yadhu Krishna	PRC22CSOT016
Nandana Biju	PRC22CSOT017
Sania Sajan John	PRC22CSOT022

Project Guide

(Signature with Date)
Ms. Salitha M K,
Assistant Professor,
Department of Internet of Things and Cyber
Security ,
Providence College of Engineering.

Project Coordinator

(Signature with Date)
Ms. Salitha M K,
Assistant Professor,
Department of Internet of Things and Cyber
Security ,
Providence College of Engineering.

Head of the Department

(Signature with Date)
Ms. Salitha M K,
Assistant Professor & HOD,
Department of Internet of Things and Cyber Security ,
Providence College of Engineering

DECLARATION

We, hereby declare that the Project Report Phase II entitled **AI CRICKET COACH**, submitted to APJ Abdul Kalam Technological University, Kerala, in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Internet of Things and Cyber Security , is a bonafide record of the project work carried out by us under the guidance of Ms. Salitha M K, Assistant Professor, Department of Internet of Things and Cyber Security , Providence College of Engineering, Chengannur.

This project work is an original piece of research and design, undertaken by us during the academic year 2025–2026, and has not been submitted previously in part or full for the award of any degree, diploma, or other qualification at any university or institution.

We further affirm that due acknowledgement has been made wherever contributions from other sources have been used in the preparation of this report.

Place: Chengannur

Date: 19-03-2026

Signature of the Students

Abiya Mariyam Alex

PRC22CSOT003

M Yadhu Krishna

PRC22CSOT016

Nandana Biju

PRC22CSOT017

Sania Sajan John

PRC22CSOT022

Batch: 2022–2026

Department of Internet of Things and Cyber Security

Providence College of Engineering, Chengannur

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. Saju P John, Principal, Providence College of Engineering, Chengannur, for providing us with the opportunity and necessary facilities to carry out the project work successfully.

We extend our heartfelt thanks to Ms. Salitha M K, Head of the Department, Department of Internet of Things and Cyber Security , for her valuable guidance, constant encouragement, and support throughout the course of this project.

We are deeply indebted to our Project Guide, Ms. Salitha M K, Assistant Professor, Department of Internet of Things and Cyber Security , for the consistent supervision, technical guidance, and constructive suggestions that helped us to complete this project report in its present form.

We also wish to thank the Project Coordinator, Ms. Salitha M K, Assistant Professor, Department of Internet of Things and Cyber Security , and all the faculty members of the Department of Internet of Things and Cyber Security for their cooperation and insightful comments during the project evaluation and presentation.

We gratefully acknowledge the support of the college management and technical staff for providing the infrastructure and resources required for the successful completion of this work.

Finally, we wish to express our profound appreciation to our family and friends for their constant moral support, encouragement, and inspiration throughout the completion of this project report.

Place: Chengannur

Date: 19-03-2026

Signature of the Students

Abiya Mariyam Alex	PRC22CSOT003
M Yadhu Krishna	PRC22CSOT016
Nandana Biju	PRC22CSOT017
Sania Sajan John	PRC22CSOT022

Batch: 2022–2026

Department of Internet of Things and Cyber Security

Providence College of Engineering, Chengannur

ABSTRACT

AI Cricket Coach is an AI-powered cricket batting analysis system designed to automatically classify batting shots and provide instant performance feedback using deep learning and computer vision techniques. A custom dataset of 8000+ labeled images was collected and categorized into four major batting shots along with an additional unknown category to improve model robustness. The images were preprocessed and used to train a YOLOv8 (You Only Look Once version 8) deep learning model in Google Colab, achieving high classification accuracy. The trained model is deployed on a Python Flask-based backend server, which performs real-time inference. An Android mobile application developed using CameraX and Retrofit captures the player's batting image and sends it to the server through a client-server architecture. The server processes the image, predicts the shot category, generates an annotated output image, and returns the result in JSON format. The Android application then displays the prediction visually and provides audio feedback using Text-to-Speech. This system enhances training efficiency by offering automated, consistent, and quick feedback without relying entirely on manual observation. It reduces human error, improves accessibility to sports analytics, and demonstrates the practical integration of YOLOv8-based computer vision with mobile application development for intelligent sports performance analysis.

Index Terms: Artificial Intelligence, Deep Learning, Computer Vision, Cricket Shot Classification, Android Application, Client-Server Architecture.

Contents

Certificate	ii
Acknowledgement	iv
Abstract	v
List of Figures	ix
List of Tables	x
List of Acronyms	xi
List of Symbols	xii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	2
1.3 Objectives	2
1.4 Scope of Study	3
2 Literature Review	5
3 System Analysis	9
3.1 Functional Requirements	9
3.2 Non-functional Requirements	11
3.3 Feasibility Analysis	11
3.3.1 Technical Feasibility	11
3.3.2 Operational Feasibility	12
3.3.3 Economic Feasibility	12
3.4 Hardware Requirements	12
3.5 Software Requirements	13
3.6 Life Cycle Used	13
3.7 Software Cost Estimation	14
3.8 Total Product Cost Estimation	15

3.9	Project Schedule using Gantt chart	15
4	Methodology	16
4.1	Proposed System	16
4.2	Android App	17
4.3	Image Data Collection	18
4.4	Data Preprocessing	18
4.5	Shot Classification Process	19
4.6	Shot Estimation Process	20
4.7	Performance Evaluation	21
4.8	Feedback Mechanism	22
4.9	Model Architecture and Training Strategy	23
4.10	Client–Server Communication and API Integration	23
5	System Design	25
5.1	Architecture Diagram	25
5.2	Flow Chart	26
5.3	Use Case Diagram	28
5.4	Activity Diagram	29
5.5	Sequence Diagram	30
5.6	Collaboration Diagram	32
6	System Implementation	33
6.1	Development Environment and Technology Stack	33
6.2	Dataset Implementation and Preparation	33
6.3	YOLOv8 Model Implementation	34
6.4	Feature Extraction and XGBoost	34
6.5	Backend Server Implementation	35
6.6	Android Application Implementation	35
6.7	Client–Server Integration	36
6.8	Testing and Validation	36
6.9	Deployment and Execution	36
7	Testing	37
7.1	Testing	37
7.1.1	Types of Testing	37
7.1.2	Test Summary Report	38

8	Results	40
8.1	Software Module	40
8.1.1	Login Module – AI Cricket Coach	40
8.1.2	Server Configuration and Logout Module – AI Cricket Coach . .	41
8.1.3	Detection Result – CUT SHOT	41
8.1.4	Detection Result – PULL SHOT	42
8.1.5	Detection Result – SWEEP	43
8.1.6	Detection Result – STRAIGHT DRIVE	44
8.1.7	Detection Result – UNKNOWN	45
8.1.8	Detection Result – NO PERSON DETECTED	46
8.2	Performance Evaluation Results	47
9	Conclusion	49
	Appendix	50
	Bibliography	52

List of Figures

3.1	Agile Model	14
3.2	MDLC Model	14
3.3	Project Schedule using Gantt chart	15
4.1	Block Diagram	17
5.1	Architecture Diagram	25
5.2	Flow Chart	27
5.3	Use Case Diagram	28
5.4	Activity Diagram	29
5.5	Sequence Diagram	31
5.6	Collaboration Diagram	32
8.1	Login Page	40
8.2	Logout Page	41
8.3	Cut Shot	42
8.4	Pull Shot	43
8.5	Sweep	44
8.6	Straight Drive	45
8.7	Unknown Stance	46
8.8	No Person Detected	47

List of Tables

3.1	COCOMO model coefficients	14
4.1	Batting Shot Categories	20
7.1	Decision Based Testing	38
7.2	Test Cases for Detection Module	39
8.1	Performance Metrics for Cricket Shot Classification Model	47

List of Acronyms

AI	Artificial Intelligence
ML	Machine Learning
CV	Computer Vision
COCOMO	Constructive Cost Model
YOLO	You Only Look Once
YOLOv8	You Only Look Once Version 8
CNN	Convolutional Neural Network
MDLC	Machine Learning Development Life Cycle
SDLC	Software Development Life Cycle
FR	Functional Requirement
NFR	NON-Functional Requirement
API	Application Programming Interface
GUI	Graphical User Interface
GNN	Graph Neural Network
ST-GCN	Spatio- Temporal Graph Convolutional Network
AGE	Angular Encoding
FPS	Frames Per Second
EDA	Exploratory Data Analysis
LoC	Lines of Codes
PM	Person-Month
OS	Operating System
IDE	Integrated Development Environment

DB	Database
REST	Representational State Transfer
HTML	HyperText Markup Language
JS	JavaScript
CSS	Cascading Style Sheets
TTS	Text To Speech
VR	Virtual Reality
AR	Augmented Reality
VAR	Video Assistant Referee
PETE	Physical Education Teacher Education
3D	Three-Dimensional
2D	Two-Dimensional

List of Symbols

$P(class_i)$	Probability
z_i	Logit
e^{z_i}	Exponential logit
$\sum_{j=1}^n$	Summation
n	Classes

Chapter 1

Introduction

1.1 Background

Cricket is one of the most widely followed and played sports across the world, where batting technique plays a decisive role in a player's overall performance. A proper stance forms the foundation for balance, coordination, timing, and shot execution. Even minor technical errors in posture or foot positioning can significantly affect accuracy and consistency. Therefore, analyzing and improving batting technique is an essential part of cricket training.

Traditionally, batting analysis has been carried out manually by coaches through direct observation or recorded video review. Although experienced coaches can provide valuable insights, the process is often subjective and time-consuming. Feedback may vary depending on the observer's expertise, and continuous monitoring of every player is not always feasible. Additionally, advanced video analysis systems can be expensive and are not easily accessible to all training environments. With the advancement of Artificial Intelligence (AI) and Computer Vision, automated performance analysis systems have gained significant attention in recent years.

Deep learning models, particularly YOLOv8 (You Only Look Once), have shown remarkable performance in real-time object detection and image classification tasks. These models can automatically learn complex visual features such as body posture, bat position, and movement patterns from large datasets, while also enabling fast and efficient detection in real-time applications.

In this project, a comprehensive dataset consisting of over 8000 cricket batting images was used to train a deep learning model capable of classifying multiple types of batting shots. The dataset includes distinct shot categories as well as an additional unknown class to improve the system's robustness and prevent incorrect classifications. Proper preprocessing techniques such as resizing, normalization, and augmentation were applied to enhance model performance and generalization.

The trained model is deployed on a Python-based backend server, which performs real-

time inference when an image is received. An Android mobile application acts as the client, capturing the player's image using the device camera and sending it to the server through a client-server architecture. The server processes the image, detects and predicts the shot category, and returns the result along with a processed image for visualization. By integrating deep learning with mobile technology, this system provides instant visual and audio feedback to players, improving training efficiency and reducing technical errors. The project demonstrates how AI-driven solutions can enhance sports analytics, making performance evaluation more consistent, accessible, and technology-driven in modern cricket training environments.

1.2 Problem statement

Cricket batting training today still relies heavily on manual observation and subjective evaluation by human coaches. This traditional approach leads to several challenges:

- Lack of immediate and automated feedback during practice sessions.
- Inconsistent analysis depending on the coach's experience and expertise.
- Difficulty in accurately classifying and quantifying different batting shots.
- Time-consuming video review processes without intelligent automation.
- Limited access to advanced performance analysis tools in many training environments.
- Dependence on costly equipment or complex setups for detailed motion analysis.

Therefore, there is a strong need for a cost-effective, AI-powered cricket batting analysis system capable of automatically classifying shots, providing real-time feedback, and delivering consistent performance evaluation through an accessible mobile application.

1.3 Objectives

The main objectives of the AI-Powered Cricket Batting Analysis System are:

1. To Develop a Large-Scale Image Dataset: To collect and organize a comprehensive dataset of 8000+ cricket batting images representing multiple shot categories along with an unknown class to ensure robust model training.
2. To Perform Image Preprocessing and Augmentation: To apply techniques such as

resizing, normalization, and data augmentation (rotation, flipping, brightness adjustment) to improve model accuracy and generalization.

3. To Design and Train a Deep Learning Model: To build and train a You Only Look Once(YOLOv8) model in Google Colab capable of accurately classifying different cricket batting shots.
4. To Implement an Unknown Shot Detection Mechanism: To enable the system to identify and label unfamiliar or incorrect batting poses as “Unknown” to prevent misclassification.
5. To Develop a Backend Server for Model Deployment: To create a Python Flask-based server in VS Code that loads the trained model and performs real-time image inference.
6. To Build an Android Client Application: To design and develop an Android application using CameraX and Retrofit that captures player images and communicates with the backend server through a client–server architecture.
7. To Provide Real-Time Visual and Audio Feedback: To display predicted shot results along with processed image overlays and deliver voice feedback using Text-to-Speech for improved user interaction.
8. To Create a Cost-Effective and Accessible Coaching Tool: To design a scalable and user-friendly system that makes AI-powered cricket training accessible without the need for expensive hardware or complex motion-tracking equipment.

1.4 Scope of Study

The scope of this project focuses on developing an AI-powered cricket batting analysis system that can automatically classify different batting shots from captured images and provide real-time feedback through a mobile application. The system is designed to work within a client–server architecture, where the Android application captures images and the backend server performs deep learning–based prediction.

The project covers the collection and preprocessing of a large dataset of 8000+ images representing multiple batting shot categories along with an unknown class. It includes model training using deep learning techniques, deployment of the trained model on a Flask-based server, and integration with an Android application for practical use.

This system is primarily intended for training and educational purposes, helping players

improve their batting technique through automated analysis. It can be used in cricket academies, schools, and individual practice sessions. The project demonstrates the practical application of Artificial Intelligence, Computer Vision, and mobile development in sports analytics.

In the future, the scope can be extended to real-time video analysis, additional shot categories, player performance tracking, cloud deployment, and integration with advanced pose estimation systems to provide more detailed biomechanical feedback.

Chapter 2

Literature Review

The complexity of human movement in sports has driven growing research in posture detection and correction. Earlier methods based on manual observation and sensors have evolved into AI and deep learning approaches, enabling more accurate and real-time exercise monitoring.

Musa Çankaya and Fatma Nur Takı [1] conducted a comparative study on postural assessment and awareness in individuals receiving posture training through a digital AI-based system. Their research highlighted that traditional posture correction methods often fail to provide consistent results, while digital AI systems offer personalized and adaptive training feedback. The study demonstrated that AI-based posture detection helps participants maintain correct body alignment during daily activities. They found that AI-driven feedback mechanisms significantly improved posture awareness among individuals who underwent training. Furthermore, the system reduced the risk of musculoskeletal disorders by correcting harmful sitting and standing postures early. The findings strongly supported the idea that integrating AI into posture correction has the potential to replace conventional methods. Their conclusion emphasized that AI-based posture monitoring tools are not only more efficient but also scalable for use in occupational health and rehabilitation.

Engdahl, C et al. [2] proposed a system to enhance the teaching of creative aspects of dance in Teacher Education (PETE). Their work emphasized that fostering creativity is a globally recognized educational objective and identified expressive dance as an effective medium to develop students' creativity. However, they observed that traditional dance education in schools and PETE often focuses on reproducing pre-arranged movement sequences and achieving predetermined outcomes, which limits opportunities for creative exploration and problem-solving. Using the theoretical framework of and, specifically the concepts of smooth and striated spaces and experimentation, Engdahl et al. analysed interviews with eight PETE dance educators from different institutions in. Their findings showed that while creativity is mentioned in PETE curricula, it remains marginal in practice due to the dominant emphasis on technical correctness and physical performance in. They concluded that teacher educators play a crucial role in shaping how future PE teachers incorporate creativity into their teaching, and suggested that encouraging exploratory tasks, improvisation, problem-solving, and the use of music are effective strategies to pro-

mote creative movement learning in expressive dance education.

Fan et al. [3] proposed a dual-source deep neural network that combines local appearance and holistic view for human pose estimation. Their framework was designed to handle the limitations of single-source models, which often struggled with occlusion and complex backgrounds. The integration of local body part features with global structural context significantly improved recognition accuracy. The model was tested in challenging environments, where it consistently outperformed existing benchmarks. By fusing local and holistic information, the network could better distinguish between similar poses with minimal errors. This dual-source approach proved particularly effective in sports and fitness applications, where body movement analysis requires high precision. The results indicated that deep learning frameworks could revolutionize the field of human pose estimation by ensuring robustness across varying conditions. The authors concluded that their framework laid the foundation for future improvements in pose estimation technology. From our analysis, we found that there is no integrated automated system that effectively combines real-time cricket performance analysis, training feedback, and posture correction using Artificial Intelligence in an efficient and accessible manner.

Felzenszwalb and Huttenlocher [4] introduced pictorial structures for object recognition, which laid a foundational framework for modern posture estimation methods. Their approach represented objects as collections of parts connected by spatial relationships, making it one of the earliest structured models in computer vision. This methodology proved highly effective for representing human body structures in posture recognition. Although their work was not originally intended for sports or exercise applications, it provided the theoretical groundwork for subsequent AI-based pose estimation research. Their model also influenced the development of part-based human detection systems that became critical in posture monitoring. The study showed that structured approaches are highly versatile, as they balance local accuracy with global structural consistency. The impact of their research continues to be visible in recent AI-driven posture detection frameworks.

Qin Z et al. [5] proposed a novel method for Skeleton-based action recognition by introducing higher-order angular features into Graph neural networks (GNNs) to improve recognition accuracy. They noted that skeleton-based action recognition is more robust to background interference and easier to process, and that recent advances in GNNs have significantly boosted its performance by modeling skeletons as graphs—where joints are nodes and bones are edges. Earlier methods used only first-order joint coordinate features, and later second-order bone vector features, which improved performance but still struggled to distinguish actions with similar motion trajectories. To overcome this limitation, Qin et al. proposed an angular encoding (AGE) approach that captures relative

movements between body parts through angles formed by three related joints, providing invariance to different body sizes and motion speeds. They demonstrated that integrating angular features into existing models such as Spatio-temporal graph convolutional network (ST-GCN) and decoupling GCN improves the classification of confusingly similar actions. Experimental results showed that their method achieved state-of-the-art accuracy on benchmark datasets like NTU60 and NTU120.

Supriya Shrivastav et al. [6] developed an AI-based exercise posture recognition system using an estimation framework. Their work focused on real-time analysis, enabling immediate detection and correction of incorrect exercise postures. The system was capable of identifying a wide range of common exercises, making it suitable for fitness coaching and rehabilitation. The researchers highlighted the importance of accurate posture detection in preventing injuries during workouts. Their model demonstrated a high recognition rate and adaptability across diverse body types and exercise forms. By integrating AI-based feedback, users were able to refine their techniques without the need for constant trainer supervision. The study showed that the framework not only enhanced training efficiency but also promoted safer workout practices. Their results emphasized that such systems have strong potential in the fitness industry, especially for at-home and remote exercise programs.

Tang T and Hyun-Joo M [7] proposed a sports dance movement detection system based on Pose recognition to enhance the learning and analysis of human actions and behaviours. They emphasized that pose recognition can standardize movements during dance teaching and preserve vital motion data, unlike traditional training methods that rely on repeated video viewing or coach feedback. Their system follows three main stages data acquisition and preprocessing, feature extraction and construction, and action recognition and addresses the low accuracy of existing methods by selecting key skeletal points and using a residual block based detection method. It captures complete dance movement data through posture recognition and bone tracking, compares it with standard actions to detect deviations, and helps learners correct nonstandard postures. The system also supports flexible, location-independent training, promoting the digitalization of Sports Dance education.

Tian et al. [8] explored the integration of Big Data and Artificial Intelligence in football to enhance training, competition, and refereeing. The study highlighted how data analytics and AI-driven tools such as VAR, GoalControl, and Scout Advisor improve tactical decision making, player evaluation, and match fairness. The authors also emphasized the role of VR, AR, and 3D reconstruction in creating immersive training environments. However, challenges such as high implementation costs, privacy concerns, and lack of

standardization were noted. The paper concluded that combining Big Data and AI provides a scientific and intelligent framework for modern football development, enabling data-driven coaching, injury prevention, and real-time performance optimization.

Wang and Zhang [9] explored AI-powered athletic posture detection in sports training through empirical research. They argued that traditional coaching methods lack precision in monitoring posture and movements, especially in high-speed athletic activities. By incorporating AI algorithms, their system was able to deliver real-time feedback to athletes, allowing for immediate corrections. Their study revealed that the AI framework significantly enhanced the efficiency of training by reducing repetitive mistakes in postures. Moreover, the system could detect subtle deviations that were often overlooked by human trainers. This led to better performance outcomes, as athletes could fine-tune their techniques with data-driven insights. The results highlighted that AI-based monitoring is crucial for modern sports training, where even small posture errors can impact performance. Ultimately, their findings stressed the importance of merging technology with sports science for achieving peak athletic performance.

Yang, X [10] proposed a dance teaching system based on to enhance the evaluation and prediction of students' dance performance. In this study, a special performance prediction model was developed where the physical form quality index was considered as the independent variable and the special achievement was taken as the dependent variable. Through association analysis and cluster analysis, six specific items with higher sensitivity related to body shape, physical quality, and special quality were identified. A model was then adopted to build the prediction framework, and its reliability and error were analysed and compared with a traditional regression model. The results showed that the neural network-based model achieved higher accuracy in predicting dance performance and assessing the development level of physical quality, demonstrating its effectiveness in supporting data-driven dance teaching systems.

Chapter 3

System Analysis

The system analysis phase involves examining the functional and non-functional requirements, design constraints, feasibility aspects, and hardware/software needs of the proposed system. This chapter provides a detailed analysis of the AI-powered Cricket Batting Analysis System, a real-time image classification and feedback system that utilizes deep learning and computer vision techniques to improve cricket training efficiency, accuracy, and accessibility through a client–server architecture.

3.1 Functional Requirements

The proposed AI-Powered Cricket Batting Analysis System is expected to meet the following functional requirements:

FR 1: Capture Player Image

Description: The system should capture a clear image of the player’s batting stance using the Android mobile camera integrated through CameraX. The application must provide a live preview and allow the user to focus before capturing the image.

Input: Live camera feed from the mobile device.

Output: High-quality captured batting image stored temporarily in device storage.

FR 2: Preprocess Captured Image

Description: The system should preprocess the captured image to make it suitable for deep learning model inference. This includes resizing the image to the required dimensions (e.g., 224×224), correcting orientation, and normalizing pixel values.

Input: Raw captured image file.

Output: Preprocessed image ready for transmission and model prediction.

FR 3: Upload Image to Backend Server

Description: The Android application should securely transmit the preprocessed image to the backend server using a multipart HTTP POST request through Retrofit. The system must support dynamic server IP configuration to ensure flexibility.

Input: Preprocessed image file.

Output: Successful image upload to the server endpoint.

FR 4: Perform Shot Classification

Description: The backend server should load the trained deep learning model and perform classification on the received image. The system must accurately identify the batting shot category or label it as unknown if it does not match trained classes.

Input: Uploaded batting image.

Output: Predicted shot category with confidence score.

FR 5: Generate Annotated Output

Description: After prediction, the server should generate a processed or annotated image (if applicable) and convert it into Base64 format for easy transmission back to the client application.

Input: Model prediction result.

Output: Base64-encoded processed image along with classification label in JSON format.

FR 6: Display Visual Feedback

Description: The Android application should receive the server response and display the predicted shot category clearly on the screen along with the processed image overlay. The UI should be responsive and user-friendly.

Input: JSON response from server.

Output: Visual representation of prediction result.

FR 7: Provide Audio Feedback

Description: The system should use Text-to-Speech functionality to announce the predicted shot category, ensuring accessibility and interactive feedback for users during practice sessions.

Input: Predicted shot label.

Output: Clear voice output informing the user of the classification result.

FR 8: User Authentication (Login and Logout)

Description: The system should allow users to securely log in using their email and password to access the AI Cricket Coach application and provide a logout option to end the session safely.

Input: User email address and password.

Output: Successful login grants access to the application, while logout securely ends the session and returns to the login screen.

3.2 Non-functional Requirements

The proposed AI Cricket Coach system must also satisfy the following non-functional requirements to ensure reliability and user satisfaction.

NFR 1: Availability: The system should be available for the user 24×7 without failure.

NFR 2: Reliability: The system should be reliable with failure free operations. The reliability value ranges from 0 to 1.

NFR 3: Maintainability: The system should be maintainable without changing the base structure.

NFR 4: Security: The system should not be vulnerable to the attackers. The data gathered and stored by the system should be secure.

3.3 Feasibility Analysis

Feasibility analysis evaluates whether the proposed AI-Powered Cricket Batting Analysis System is practical, implementable, and beneficial within given constraints. It examines the technical capability, operational practicality, and economic viability of developing and deploying the system. This ensures that the project can be successfully implemented using available resources and technologies.

3.3.1 Technical Feasibility

Technical feasibility assesses whether the required technology, tools, and technical expertise are available to develop the system. The proposed system is technically feasible as it utilizes widely available and proven technologies such as Python, Flask for model training, and Android Studio for mobile application development. The deep learning model was successfully trained using Google Colab, which provides sufficient computational power for model development. The backend server can be executed in VS Code using standard hardware with moderate specifications. The Android application uses CameraX for image capture and Retrofit for server communication, both of which are well-supported and stable libraries. Since the system follows a client–server architecture, heavy computations are performed on the server, reducing the processing burden on the mobile device. Therefore, from a technological perspective, the system is practical and achievable.

3.3.2 Operational Feasibility

Operational feasibility determines whether the system will function effectively in real-world conditions and whether users can operate it easily. The system is designed to be user-friendly and accessible. Players simply need to open the Android application, capture an image, and receive instant feedback. No specialized technical knowledge is required to operate the system. The interface is simple, interactive, and provides both visual and audio feedback to enhance usability. The solution can be used in cricket academies, schools, or personal practice sessions. Since the system works with standard smartphones and does not require expensive motion-tracking hardware, it is practical for everyday training use. Thus, the system is operationally feasible and suitable for real-world deployment.

3.3.3 Economic Feasibility

Economic feasibility evaluates whether the project is cost-effective and financially viable. The proposed system is economically feasible because it uses open-source tools and platforms such as Python, Flask, and Android Studio. Model training was performed using Google Colab, which offers free computational resources. The system does not require expensive sensors, motion-capture devices, or specialized hardware. Only a standard smartphone and a basic computer for server deployment are needed. This significantly reduces overall implementation cost compared to traditional motion-tracking or advanced sports analytics systems. Therefore, the project provides a cost-effective solution for AI-based cricket training.

3.4 Hardware Requirements

To ensure optimal performance and seamless functionality of the AI-Powered Cricket Batting Analysis System, certain hardware components are essential. The following hardware resources were utilized for system development, training, deployment, and testing:

- Laptop or Desktop Computer (minimum 8GB RAM, Intel i5 or equivalent processor) for model training, server execution, and development in VS Code
- Android Smartphone (with minimum Android 8.0 and functional rear camera) for running the mobile application
- High-Resolution Smartphone Camera (minimum 8MP recommended) for capturing clear batting images
- Stable Internet Connection (Wi-Fi or Mobile Data) for client–server communication

and real-time image upload

- Cloud-Based Platform (Google Colab) for deep learning model training using GPU resources
- External Storage (optional) for dataset storage containing 8000+ images
- Audio Output Device (mobile speaker or headset) for Text-to-Speech feedback

These hardware components ensure smooth image capture, efficient server communication, accurate model inference, and proper feedback delivery during cricket training sessions.

3.5 Software Requirements

The primary software tools and technologies used in the development of the AI-Powered Cricket Batting Analysis System include:

- Operating System: Windows / macOS / Linux
- Programming Languages: Python (Backend and Model), Java (Android Application)
- Frameworks: Flask (Backend Server), Android SDK
- AI Libraries: YOLOv8, XGBoost
- Deep Learning Platform: Google Colab (for model training with GPU support)
- Networking Library: Retrofit (for client-server communication)
- Database/Storage: Local storage for dataset (optional cloud storage for backup)
- Development Environment: Visual Studio Code (Server), Android Studio (Mobile App), Google Colab (Model Training)

These software components collectively support dataset preparation, model training, back-end deployment, mobile application development, and real-time shot classification functionality.

3.6 Life Cycle Used

In this project, we chose the Agile model. It is an iterative and incremental development approach. We develop our project as different modules, which are completed in multiple

sprints. The Agile model shown in Figure 3.1 is flexible and makes it easier to incorporate new features and improvements during the development phase.

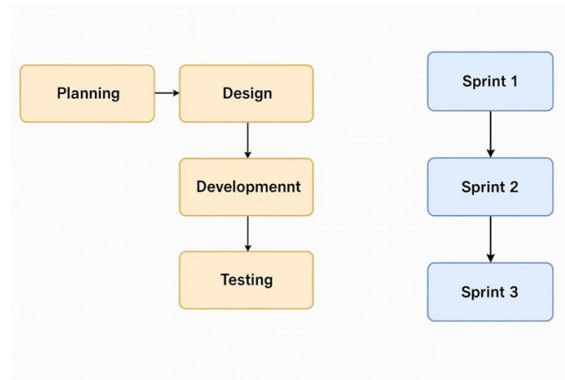


Fig. 3.1. Agile Model

Analogous to the role of the software-development lifecycle (SDLC), the machine learning model-development lifecycle (MDLC) guides the activities of ML model development from inception through retirement. The key phases of the MDLC including data ingestion, exploratory data analysis, model creation, and model operation are illustrated in Figure 3.2.

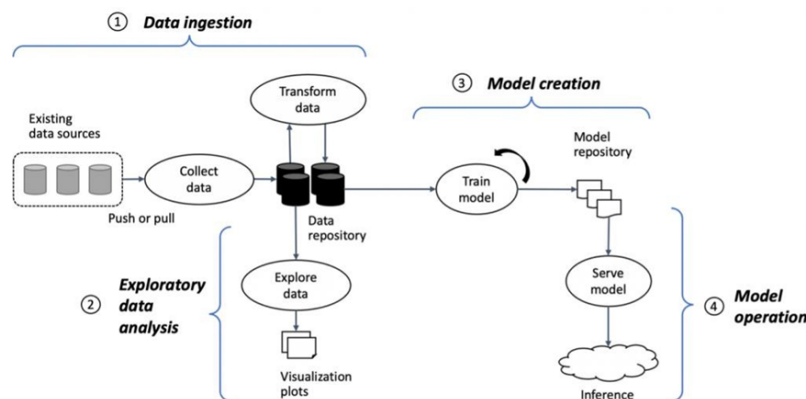


Fig. 3.2. MDLC Model

3.7 Software Cost Estimation

Table 3.1: COCOMO model coefficients

Software Category	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Software Project Category: Organic

Estimated Lines of Code (LOC): 3500 LOC = 3.5 KLOC

Effort applied = $a \cdot (KLOC)^b$ [Person-Months] = $2.4(3.5)^{1.05} = 8.94$ PM (Person-Months)

Development Time = $c \cdot (\text{Effort})^d = 2.5(8.94)^{0.38} = 5.74$ months / 175 days

No: of Persons = Effort / Development Time = $8.94 / 5.74 = 1.55 = 2$ person

3.8 Total Product Cost Estimation

The total product cost is the sum of both hardware and software costs involved in developing the AI Cricket Coach system. As this project uses certain premium tools and cloud-based services like Google Colab Pro for AI model training and testing.

The estimated cost for developing this project is:

Total cost = Hardware cost + Software cost = $0 + 6000 = 6000$ INR

3.9 Project Schedule using Gantt chart

ID	TASK NAME	Start	Finish	Duration	2025					2026	
					Aug	Sep	Oct	Nov	Dec	Jan	Feb
1	Requirement Gathering	01/08/2025	05/08/2025	4d	■						
2	Literature Survey	06/08/2025	20/08/2025	13d	■	■					
3	Feasibility Analysis	21/08/2025	30/08/2025	9d		■					
4	System Modelling and Design	01/09/2025	20/09/2025	18d		■	■				
5	Software Implementation	15/09/2025	15/11/2025	68d			■	■	■		
6	ML Model Development	10/10/2025	29/11/2025	44d			■	■	■		
7	Android App Development	01/11/2025	20/12/2025	43d				■	■	■	
8	Unit Testing	15/12/2025	10/01/2026	24d					■	■	
9	Integration and Unit Testing	12/01/2026	14/02/2026	30d						■	■
10	Deployment	20/02/2026	21/02/2026	2d							■
11	Documentation	01/08/2025	28/02/2026	182d	■						

Fig. 3.3. Project Schedule using Gantt chart

The Gantt chart shows the project scheduling. Gantt chart shows the start and finish date of the project. Our project phases will be completed as per the prescribed time schedule in the above Gantt chart shown in Figure 3.3. The starting date was 01/08/2025 and the project is expected to complete on 28/02/2026.

Chapter 4

Methodology

4.1 Proposed System

The proposed AI-Powered Cricket Batting Analysis System follows a structured methodology that integrates deep learning, computer vision, and a client–server mobile architecture to achieve real-time shot classification and feedback.

The methodology begins with dataset collection and preparation, where over 8000 cricket batting images were gathered and categorized into predefined shot classes along with an unknown class. The images were preprocessed through resizing, normalization, and data augmentation techniques to improve model performance and generalization. The dataset was then divided into training and testing sets for model development.

Next, a YOLOv8 deep learning model was trained using the Ultralytics framework in Google Colab. The model learned important visual features such as player posture, bat orientation, and body alignment to detect and classify different cricket batting shots. After achieving satisfactory performance, the trained model was exported and integrated into the backend server for real-time inference.

In the deployment phase, the trained model was integrated into a Python Flask-based backend server developed in VS Code. The server is responsible for receiving images from the Android application, preprocessing them, performing model inference, and generating a prediction result. The server then converts the processed output image into Base64 format and sends it back to the client in JSON format.

On the client side, an Android application developed using Android Studio captures the player's image through CameraX and sends it to the backend server using Retrofit. Upon receiving the server response, the application displays the predicted shot category along with the processed image and provides voice feedback using Text-to-Speech.

Thus, the proposed system methodology ensures efficient data flow from image capture to prediction and feedback, enabling real-time, automated cricket shot analysis through a reliable and scalable client–server architecture.

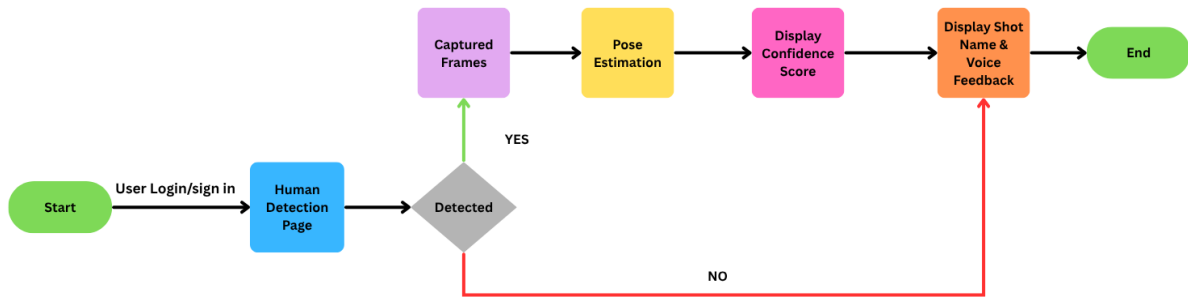


Fig. 4.1. Block Diagram

4.2 Android App

The AI-Powered Cricket Batting Analysis system is implemented as an Android application to improve accessibility and practical usability. The mobile application enables players to capture their batting images directly using their smartphone camera during practice sessions. The captured image is transmitted to the backend server, where the trained deep learning model performs shot classification and analysis.

The Android app interface provides:

- Camera Capture Option, allowing users to capture batting images in real-time using CameraX integration.
- Server Connectivity Module, enabling image upload to the Flask backend server via Retrofit API.
- Live Prediction Display, showing the classified shot category along with the processed output image returned from the server.
- Instant Voice Feedback, using Text-to-Speech functionality to announce the predicted result.
- Server Configuration Option, allowing dynamic IP address entry for flexible client–server communication.
- Error Handling and Status Indicators, ensuring smooth user interaction during image upload and result retrieval.

This application allows players to practice anywhere using only a smartphone and internet connection, eliminating the need for expensive motion-tracking equipment and making AI-based cricket training portable, affordable, and efficient.

4.3 Image Data Collection

The data collection stage forms the foundation of the model development process. The system requires multiple image samples of each batting shot captured under consistent and controlled conditions to ensure accurate classification performance.

- **Camera Placement:** Smartphone camera positioned approximately 3–4 meters from the batsman at waist or shoulder height for clear full-body visibility.
- **Image Resolution:** Minimum 224×224 pixels after preprocessing to match model input size.
- **Lighting Conditions:** Uniform lighting to reduce shadows and improve feature extraction accuracy.
- **Background:** Simple or less cluttered background to clearly isolate the batsman's posture and bat position.
- **Camera Angle:** Side or slightly angled view to clearly capture body posture and bat swing direction.

The dataset consists of 8000+ labeled images representing four different batting shot categories along with an additional unknown class to improve generalization. Each shot type was captured multiple times with variations in posture, angle, and player style to enhance diversity.

The collected dataset was divided into:

- **Training Set** – 70% (for model learning)
- **Testing Set** – 20% (for performance evaluation)
- **Validation Set** – 10% (for tuning and accuracy improvement)

This structured data collection approach ensures better model training, improved accuracy, and robust real-world performance.

4.4 Data Preprocessing

Before model training, the collected image dataset undergoes several preprocessing steps to ensure consistency, improve model performance, and enhance generalization capability:

- **Image Resizing:** All images are resized to a fixed dimension (e.g., 224×224 pixels) to match the input size required by the YOLOv8 model and maintain uniformity.

- **Normalization:** Pixel values are scaled to a range of 0–1 to improve numerical stability and accelerate the learning process during training.
- **Data Cleaning:** Blurred, low-quality, or irrelevant images are removed to maintain dataset quality and prevent inaccurate predictions.
- **Background Handling:** Minor noise reduction techniques are applied where necessary to reduce background distractions and focus on the player's posture and bat position.
- **Data Augmentation:** Techniques such as rotation, horizontal flipping, zooming, and brightness adjustment are applied to increase dataset diversity and reduce overfitting.
- **Label Encoding:** Each image is assigned a categorical label corresponding to its shot class, including an additional unknown class to improve robustness.

These preprocessing steps ensure that the model learns important visual features such as body posture, bat angle, and stance alignment accurately, regardless of lighting conditions or background variations.

4.5 Shot Classification Process

The shot classification process identifies the type of batting shot from the captured image using a trained You Only Look Once (YOLOv8) model. Instead of extracting skeletal keypoints, the system analyzes visual posture patterns directly from the image to determine the correct category.

Using a trained deep learning model built with TensorFlow/Keras, the system processes the input image and extracts important spatial features such as body alignment, bat angle, arm extension, leg positioning, and overall stance balance. These features are learned automatically during training using the 8000+ labeled dataset.

Each input image is represented as a matrix of pixel values (after resizing and normalization). The YOLOv8 model processes the image through multiple deep learning layers to extract meaningful visual patterns that help identify different cricket batting shots.

The architecture performs the following operations:

- **Feature Extraction:** The YOLOv8 backbone network extracts important visual features such as player posture, bat direction, and body alignment.

- **Feature Aggregation:** The neck layer combines features from different scales to improve detection of relevant patterns in the image.
- **Object Detection and Classification:** The detection head predicts the location of the player and classifies the batting shot into the predefined categories.
- **Confidence Prediction:** The model outputs confidence scores for each predicted class, including the unknown category, indicating the likelihood of the detected shot.

The system selects the class with the highest probability score as the final prediction. If the prediction confidence is low, the image is classified as unknown to avoid misclassification.

This process enables accurate and efficient shot identification based purely on visual posture analysis, making the system practical for real-time AI-based cricket training.

4.6 Shot Estimation Process

After image preprocessing and feature extraction, the system identifies which batting shot the player attempted based on learned visual posture patterns. Instead of joint angle calculations, the classification is performed using a trained You Only Look Once(YOLOv8) model that recognizes distinctive body alignment, bat position, and stance characteristics.

Each shot category in the dataset is characterized by unique visual features such as bat swing direction, body posture, leg positioning, and weight distribution. These visual cues enable the model to accurately differentiate between shot types.

Table 4.1: Batting Shot Categories

Shot Category	Distinct Visual Features
Straight Drive	Characterized by a vertical bat swing with strong forward body alignment and balanced weight transfer to the front foot.
Pull Shot	Identified by horizontal bat movement with upper-body rotation and weight shift toward the back leg.
Cut Shot	Recognized by angled bat direction with lateral body movement and open shoulder alignment.
Sweep Shot	Shows lowered body posture with bent knees and bat swung close to the ground in a sweeping motion.
Unknown Class	Used when posture does not match any trained shot patterns or prediction confidence is below threshold.

The system uses a trained deep learning classifier built with TensorFlow/Keras to analyze extracted image features and categorize the input into the correct shot class. The Soft-max output layer generates probability scores for each shot, and the class with the highest probability is selected as the final prediction.

This approach ensures accurate and efficient identification of straight drive, pull shot, cut shot, and sweep shot using image-based visual pattern recognition, making the system suitable for real-time AI-powered cricket training applications.

- Automated shot classification using Artificial Intelligence and deep learning for accurate and reliable feedback.
- Real-time prediction and instant result display during batting practice sessions.
- Cost-effective virtual coaching system that eliminates the need for expensive motion-tracking equipment or professional trainers.
- Simple and user-friendly Android mobile interface for easy image capture and result viewing.
- Portable and accessible solution that allows players to practice anytime and anywhere.
- Client–server architecture that efficiently handles heavy processing on the backend server.
- Scalable system design that allows addition of new shot categories and advanced features in the future.
- Unknown shot detection mechanism to minimize misclassification and improve prediction reliability.
- Enhances overall training efficiency by providing consistent and quick performance evaluation.

4.7 Performance Evaluation

The performance evaluation process measures how accurately the system classifies the player's batting shot based on the trained deep learning model. Instead of joint-angle deviation, the evaluation is performed using classification metrics derived from model predictions.

For each test image, the model calculates a probability score (0–1) for each shot category using the Softmax output layer:

$$P(class_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Where:

- (z_i) represents the output score of class i before activation.
- (n) represents the total number of shot classes including the unknown class.

The class with the highest probability score is selected as the final prediction.

To evaluate overall model performance, the following metrics are used:

- Accuracy – Percentage of correctly classified images out of total test images.
- Precision – Measures how many predicted shots were actually correct.
- Recall – Measures how well the model identifies all actual instances of each shot.
- F1-Score – Harmonic mean of precision and recall for balanced evaluation.
- Confusion Matrix – Displays correct and incorrect predictions across all shot categories.

4.8 Feedback Mechanism

The feedback module acts as the intelligent coach interface of the AI-Powered Cricket Batting Analysis System. After the captured image is processed and the shot is classified by the deep learning model, the system delivers instant feedback to help the player understand their performance.

- Visual Display: Shows the predicted shot name clearly on the screen along with the processed image returned from the server. This allows the player to immediately verify the recognized shot.
- Confidence Indicator: Displays the prediction confidence score (if enabled), helping users understand the reliability of the classification result.
- Text-Based Feedback: Provides short on-screen messages such as “Correct Shot Detected” or “Unrecognized Shot – Try Again” when the prediction confidence is low.

- **Audio Guidance:** Uses Text-to-Speech (TTS) functionality to announce the predicted shot category, enabling hands-free feedback during practice sessions.
- **Unknown Shot Alert:** If the image does not match trained categories or falls below the confidence threshold, the system labels it as “Unknown,” preventing incorrect evaluation.

This feedback mechanism enables players to practice independently while receiving immediate AI-based guidance, thereby improving consistency, accuracy, and overall batting performance over time.

4.9 Model Architecture and Training Strategy

The core of the proposed system is a YOLOv8 (You Only Look Once version 8) model designed for multi-class image classification. The model was trained using a dataset of 8000+ labeled images representing four batting shots along with an unknown class.

The YOLOv8 architecture performs real-time object detection and classification by processing the entire image in a single forward pass. It utilizes advanced feature extraction layers along with optimized detection heads to identify and classify batting poses efficiently. The model learns important visual patterns such as bat angle, body alignment, shoulder rotation, and leg positioning, enabling accurate and fast prediction.

After feature extraction, the output is flattened and passed through Fully Connected (Dense) layers for classification. The final layer uses a Softmax activation function to generate probability scores for each shot category. The model was compiled using:

- **Loss Function:** Categorical Crossentropy (for multi-class classification)
- **Optimizer:** Adam optimizer
- **Evaluation Metric:** Accuracy

To improve generalization and prevent overfitting, techniques such as data augmentation, validation splitting, and proper epoch tuning were applied. The model was trained and tested using Google Colab with GPU acceleration, ensuring efficient training performance.

4.10 Client–Server Communication and API Integration

The system follows a structured client–server architecture, where the Android application acts as the client and the Flask-based backend server performs model inference.

When the user captures an image through the Android application, the image is temporarily stored and sent to the backend server using a multipart HTTP POST request via the Retrofit networking library. The server receives the image through a defined Flask API endpoint.

On the server side, the image undergoes preprocessing (resizing and normalization) before being passed into the trained YOLOv8 model for prediction. The model generates a probability distribution across all shot classes, and the highest probability class is selected as the final output.

The server then encodes the processed image into Base64 format and sends a structured JSON response containing:

- Predicted shot label
- Confidence score
- Encoded processed image

The Android application receives this response, decodes the image, displays the result on the screen, and provides voice feedback using Text-to-Speech. Proper error handling mechanisms are implemented to manage network failures or invalid inputs.

This structured communication ensures smooth data flow, efficient processing, and real-time feedback delivery between the mobile application and the AI model.

Chapter 5

System Design

5.1 Architecture Diagram

The proposed system model is depicted using a variety of system modelling techniques, including an architecture diagram, a use case diagram, an activity diagram, a sequence diagram, a collaboration diagram and a flow chart.

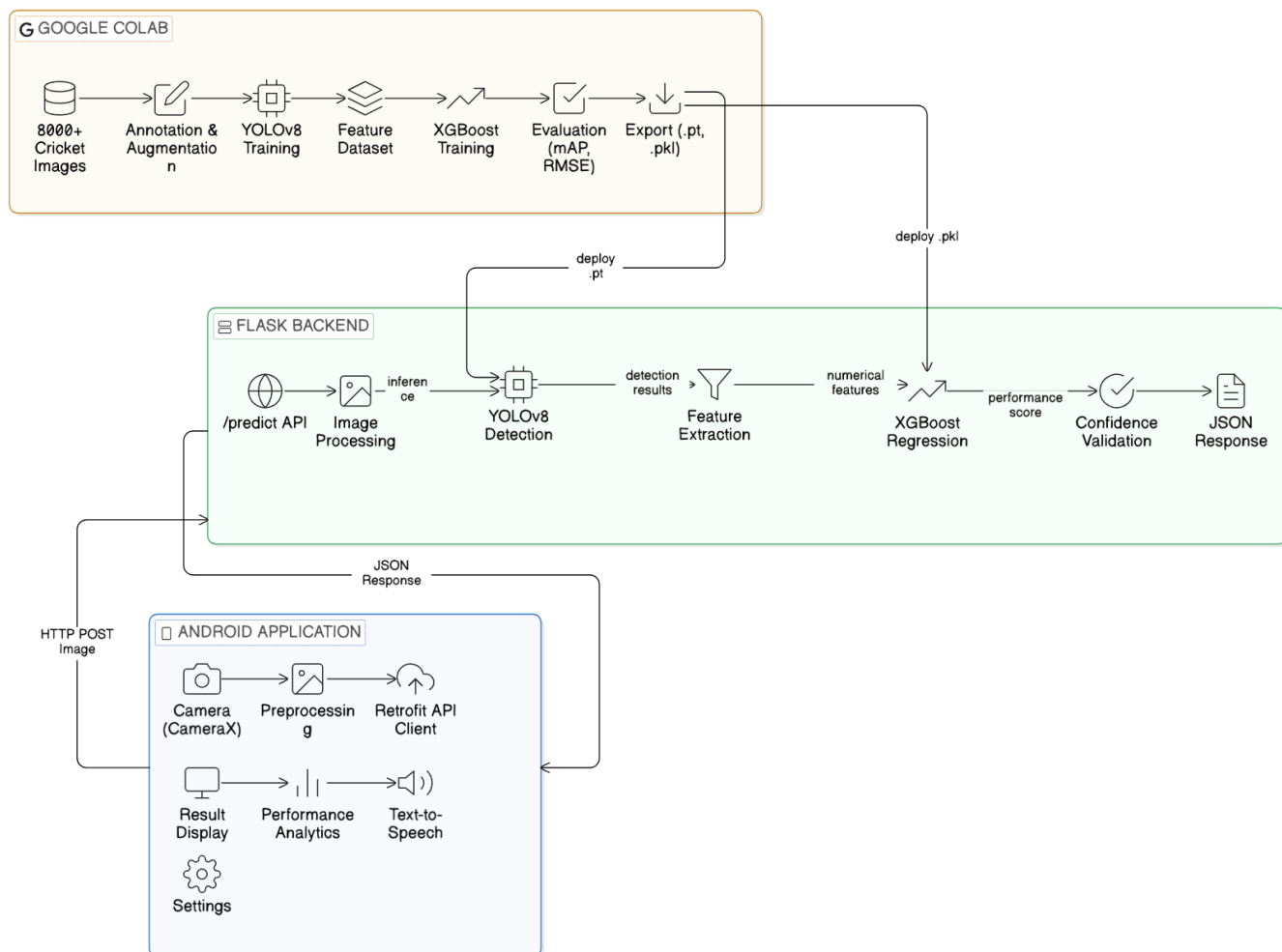


Fig. 5.1. Architecture Diagram

Description:-

The Fig.5.1. illustrates the architecture of the AI-Powered Cricket Shot Classification System consists of three main layers: the Training Environment, Client Layer, and Server Layer. In the training phase, a YOLOv8 model is developed using Google Colab with over 8000 cricket batting images, applying data augmentation, training, and evaluation before exporting the trained model as an '.h5' file for deployment. The Client Layer includes an Android application that captures batting images using CameraX, preprocesses them, and sends them to the Flask backend via an HTTP POST request. The Server Layer handles authentication, image preprocessing, and model inference using a YOLOv8 model to classify the shot as straight drive, pull shot, cut shot, or sweep. Based on a confidence threshold, the system either returns a valid prediction or marks it as unknown. The final result is sent back as a JSON response, displayed in the app, and announced through a Text-to-Speech module, enabling real-time and user-friendly shot analysis.

5.2 Flow Chart

Description:-

The Fig. 5.2. illustrates the working process of the proposed cricket shot classification system. The process begins when the user opens the Android application and captures a batting image using the device camera. The captured image undergoes initial preprocessing on the mobile device, such as resizing and normalization, to ensure it meets the required input format. The preprocessed image is then transmitted to the backend server for further analysis. Upon receiving the image, the server performs additional preprocessing steps if necessary and forwards the image to the YOLOv8 model for shot detection and classification. The YOLOv8 model extracts relevant features, identifies the batting posture, and predicts the corresponding shot along with a confidence score. The system then evaluates whether the predicted confidence score meets a predefined threshold. If the confidence is greater than or equal to the threshold, the shot is marked as a valid prediction; otherwise, it is classified as an unknown shot to avoid incorrect outputs. After classification, a structured JSON response containing the predicted shot label and confidence value is generated and sent back to the Android application. The application receives the response, displays the predicted shot on the screen, and provides voice feedback to enhance user interaction.

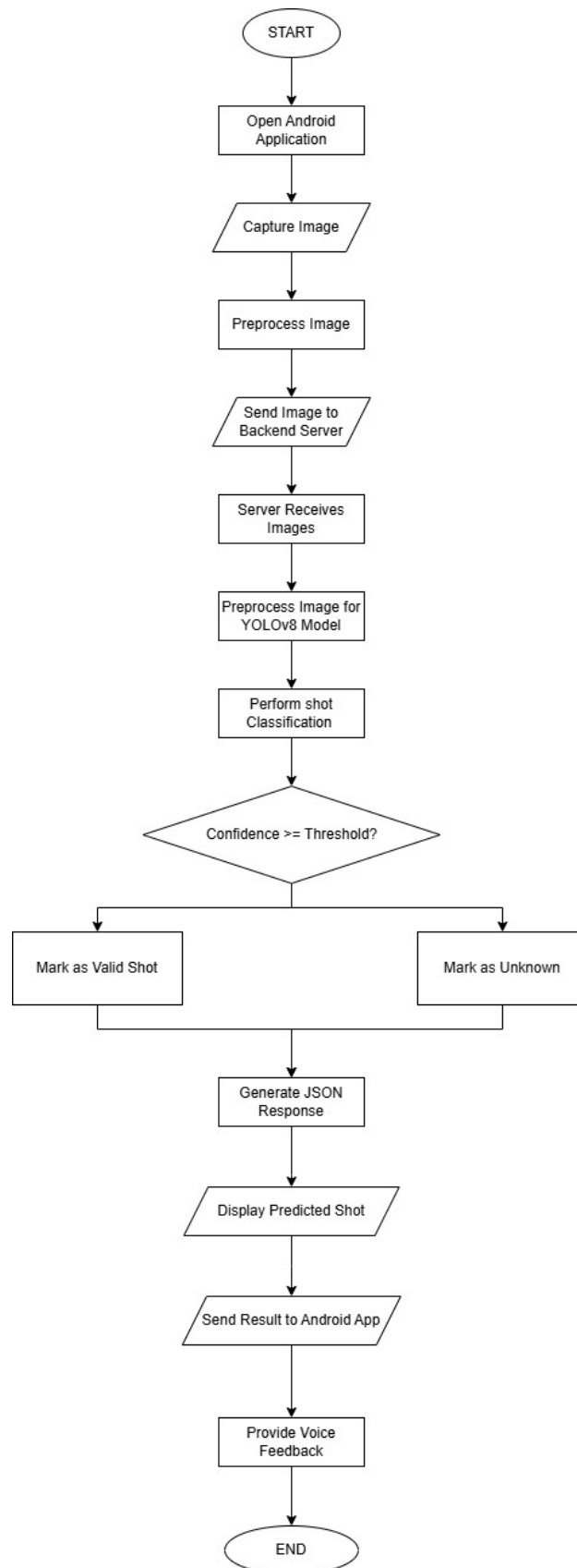


Fig. 5.2. Flow Chart

5.3 Use Case Diagram

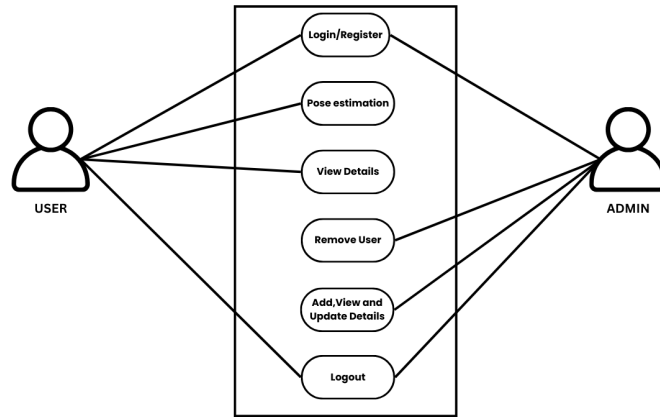


Fig. 5.3. Use Case Diagram

Description:-

Fig. 5.3 represents the Use Case Diagram of the proposed cricket shot analysis system, illustrating the interaction between two primary actors: the User and the Admin. The diagram highlights the functional requirements of the system within a defined system boundary. The User actor is associated with fundamental operations such as Login/Register, which enables secure authentication and account creation; Pose Estimation, which allows the user to upload or capture images for pose detection and shot analysis; and View Details, through which the user can access prediction results, performance insights, or related information. The Admin actor possesses higher-level privileges to manage and control the system effectively. In addition to Login/Register functionality, the Admin can Remove User accounts to maintain system integrity and handle unauthorized access. The Admin is also responsible for Add, View, and Update Details, which may include managing shot categories, updating model-related information, maintaining user records, and overseeing stored data.

5.4 Activity Diagram

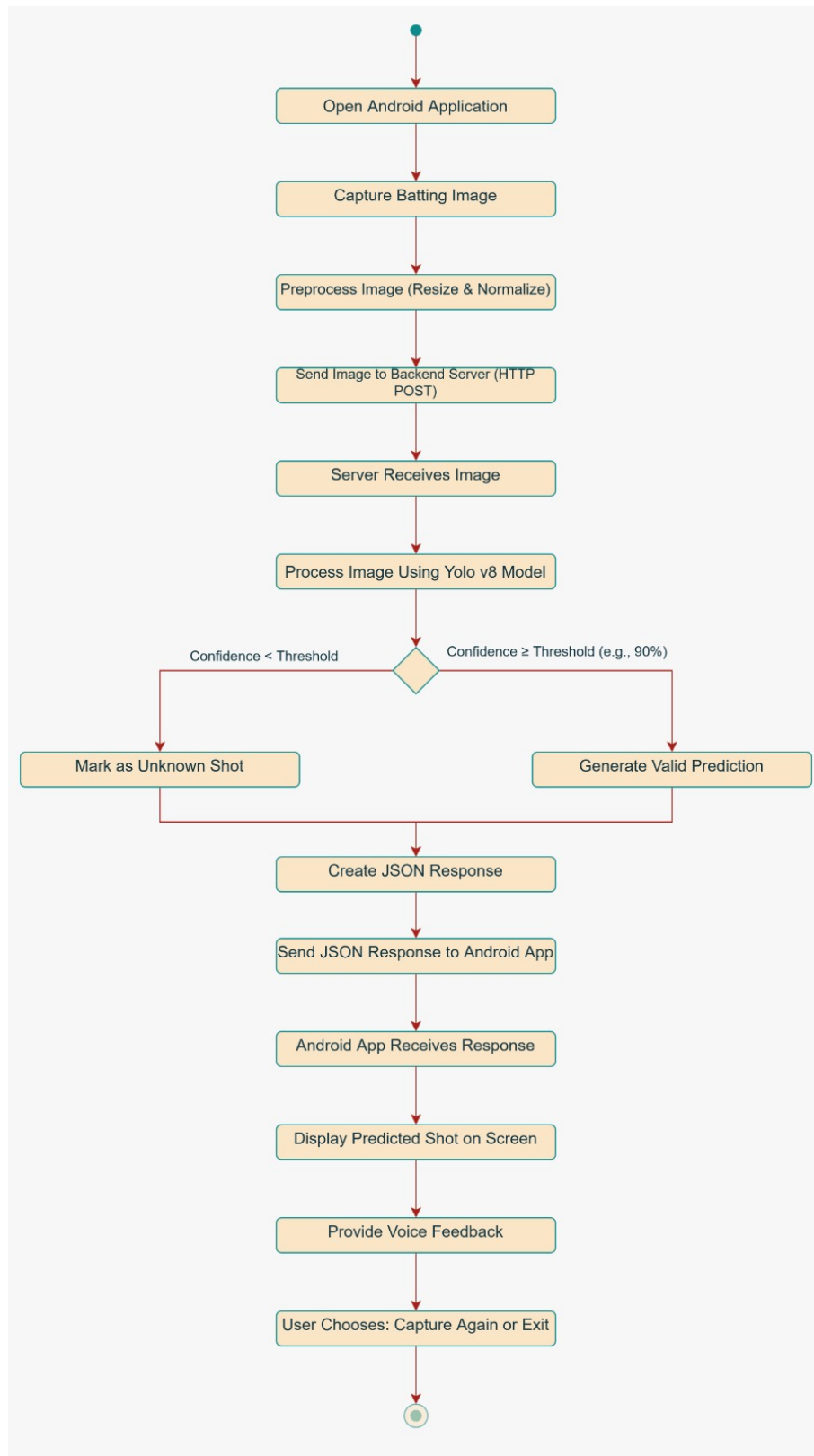


Fig. 5.4. Activity Diagram

Description:-

The Fig.5.4. is the activity diagram which illustrates the operational workflow of the Android-based cricket shot classification system. The process starts when the user opens the Android application and captures a batting image, which is then preprocessed by resizing and normalizing before being sent to the backend server via an HTTP POST request. The server receives the image and processes it using the YOLOv8 model to perform shot classification. A decision node evaluates whether the prediction confidence meets a predefined threshold (which is 90%); if the confidence is below the threshold, the shot is marked as unknown, otherwise a valid prediction is generated. The system then creates a JSON response and sends it back to the Android application, where the result is received, displayed on the screen, and supported with voice feedback. Finally, the user decides whether to capture another image or exit the application, completing the activity flow.

5.5 Sequence Diagram

Description:-

The Fig.5.5. illustrates the end-to-end workflow of the AI-Powered Cricket Shot Classification System. The process begins when the user opens the Android application and captures a batting image, which is then preprocessed (resized and normalized) within the app before being sent to the Flask backend server via an HTTP POST request. The server further preprocesses the image and forwards it to the YOLOv8 model for shot classification. The model returns a predicted label straight drive, pull shot, cut shot, or sweep along with a probability score. The server performs a confidence threshold check to determine whether the prediction is valid or should be marked as unknown. It then sends a JSON response containing the prediction and encoded image back to the Android application. Finally, the app displays the predicted result to the user and sends the label to the Text-to-Speech module, which announces the detected cricket shot.

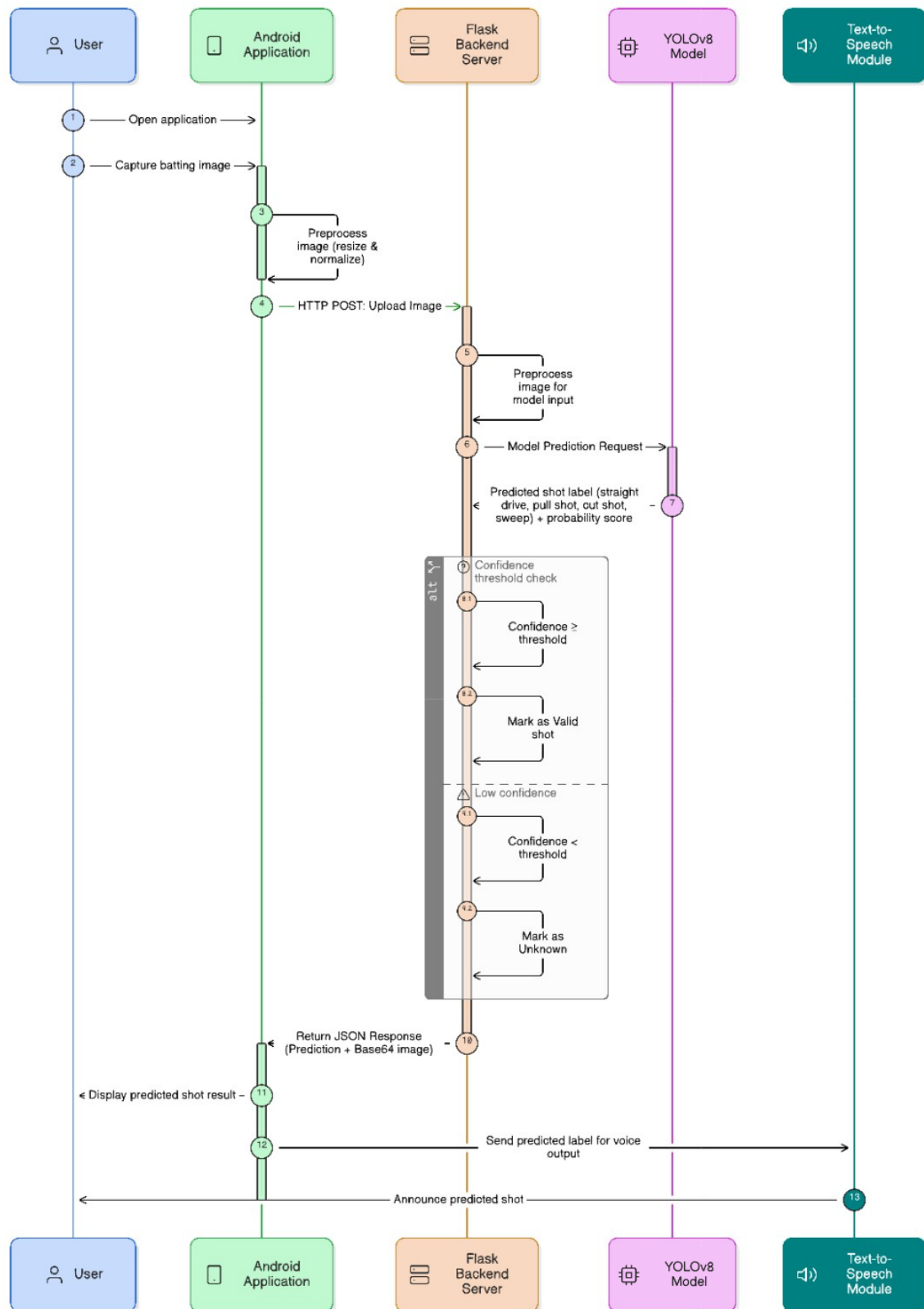


Fig. 5.5. Sequence Diagram

5.6 Collaboration Diagram

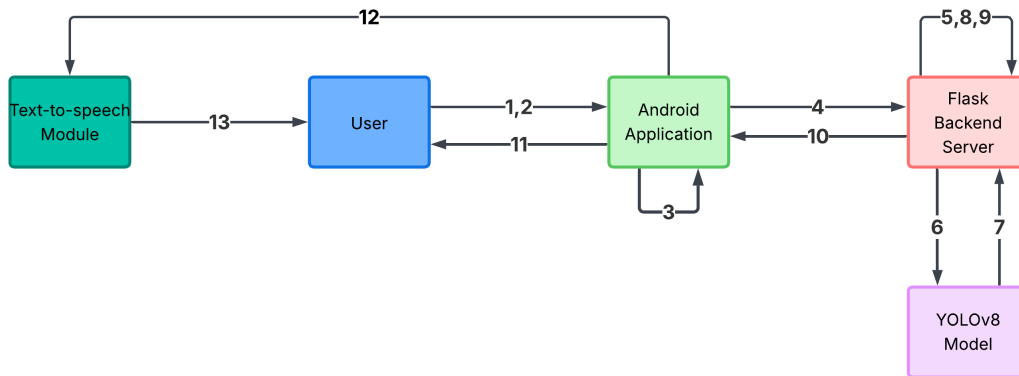


Fig. 5.6. Collaboration Diagram

Description:-

The Fig.5.6. illustrates the interaction between the User, Android Application, Flask Backend Server, YOLOv8 Model, and Text-to-Speech Module in the AI-Powered Cricket Shot Classification System. The process begins with the user opening the application and capturing a batting image through the Android app. The app preprocesses the image and sends it to the Flask backend server via an HTTP request. The server forwards the processed image to the YOLOv8 model for classification, which predicts the shot type straight drive, pull shot, cut shot, or sweep and returns a probability score. The server evaluates the confidence level, prepares a JSON response, and sends the prediction back to the Android application. The app then displays the result to the user and forwards the predicted label to the Text-to-Speech module, which announces the detected cricket shot, completing the interaction flow.

Chapter 6

System Implementation

The System Implementation phase describes the practical realization of the proposed AI-Powered Cricket Batting Analysis System. This phase involves integrating deep learning, backend development, and mobile application development into a fully functional system. The implementation was carried out in a modular manner to ensure scalability, maintainability, and efficiency.

6.1 Development Environment and Technology Stack

The AI Cricket Coach system was developed using a combination of modern software tools and frameworks to ensure high performance, flexibility, and scalability. The backend was implemented using the Python programming language along with the Flask framework to build RESTful APIs for handling communication between the mobile application and the AI models. The core detection functionality was implemented using the YOLOv8 object detection model, while XGBoost regression was integrated to generate performance scores for each cricket shot. OpenCV was used for image preprocessing and manipulation tasks before model inference. The backend development and testing were carried out in Visual Studio Code. For model training, Google Colab with GPU acceleration was utilized to speed up training and optimize computational efficiency. The Ultralytics implementation of YOLOv8 and the XGBoost library were used for training and evaluation, while NumPy and Pandas were employed for feature processing and data handling. The mobile application was developed in Android Studio using Java, integrating CameraX for image capture, Retrofit for HTTP communication, Gson for JSON parsing, and the Text-to-Speech API for voice feedback. Proper dependency management and version compatibility were maintained to avoid runtime conflicts between modules.

6.2 Dataset Implementation and Preparation

The dataset forms the foundation of the AI Cricket Coach system, as model accuracy heavily depends on data quality. The dataset consists of over 8000 labeled cricket batting images categorized into Straight Drive, Pull Shot, Cut Shot, Sweep Shot, and an

additional Unknown class to handle uncertain predictions. Each image was manually verified to ensure labeling correctness. For YOLOv8 training, bounding box annotation was performed to identify key regions such as the bat position, player body alignment, and shot execution frame. These annotations were stored in YOLO format using text files containing normalized coordinates. During preprocessing, all images were resized to a fixed resolution, typically 640×640 pixels, to match YOLOv8 input requirements. Pixel normalization was applied to improve model convergence during training. Data augmentation techniques such as horizontal flipping, rotation, brightness adjustment, and zoom scaling were used to enhance dataset diversity and reduce overfitting. The dataset was split into 70% training data, 20% testing data, and 10% validation data to ensure reliable evaluation and generalization capability.

6.3 YOLOv8 Model Implementation

The system uses YOLOv8 for real-time cricket shot detection and classification instead of a traditional CNN classifier. YOLOv8 was selected due to its fast inference speed, real-time detection capability, high mean Average Precision (mAP), and lightweight architecture suitable for deployment. The architecture consists of a backbone network for feature extraction, a neck layer for feature aggregation, and a detection head responsible for bounding box prediction and classification. During training, pretrained weights were used as a base model and fine-tuned using the custom cricket dataset. The optimizer used was either SGD or Adam, depending on performance evaluation. The training process was conducted over multiple epochs to allow the model to learn important visual features such as bat angle, arm extension, body alignment, and follow-through direction. The loss function included bounding box loss, classification loss, and objectness loss to ensure accurate localization and classification. After successful training and validation, the model was exported as a .pt file for integration into the backend server.

6.4 Feature Extraction and XGBoost

To enhance the system beyond simple classification, a regression-based performance scoring mechanism was implemented using XGBoost. After YOLOv8 detects and classifies a cricket shot, numerical features are extracted from its output, including detection confidence score, bounding box dimensions, bat angle approximation, posture alignment, and shot stability indicators. These features are structured into numerical vectors and used to train the XGBoost regression model. XGBoost was chosen due to its high prediction accuracy, ability to handle non-linear relationships, and robustness against overfitting. The

regression model predicts a performance score that reflects the quality and technical correctness of the shot. Evaluation metrics such as Root Mean Square Error (RMSE) and R-squared (R^2) score were used to assess the model's performance. The trained regression model was saved in .json or .pkl format and deployed within the Flask backend for real-time scoring.

6.5 Backend Server Implementation

The backend server functions as the core processing unit of the AI Cricket Coach system. It was developed using the Flask framework and provides a RESTful API endpoint named “/predict” to handle image prediction requests. When the Android application sends an image via HTTP POST request, the server receives the image, converts it into a NumPy array, and resizes it to match YOLOv8 input dimensions. The YOLOv8 model performs detection and classification, and relevant features are extracted from the detection results. These features are passed to the XGBoost regression model to compute a performance score. A confidence threshold mechanism is implemented to reduce misclassification; if the confidence score is below the predefined threshold, the shot is labeled as “Unknown.” The server then generates a structured JSON response containing the predicted shot type, confidence value, performance score, and feedback message, which is sent back to the Android application.

6.6 Android Application Implementation

The Android application serves as the user interface of the system and allows players to interact with the AI Cricket Coach. It was developed in Android Studio using Java. The CameraX API was integrated to enable real-time image capture of batting actions. After capturing an image, the application processes and prepares it for transmission to the back-end server. Retrofit was used to send the image as a multipart HTTP request and handle asynchronous network communication. Upon receiving the JSON response, the application parses the data using Gson and dynamically updates the user interface to display the predicted shot type and performance score. The Text-to-Speech API was integrated to provide instant voice feedback, enhancing user experience. Logout and session management features were implemented to maintain secure user access. Error handling mechanisms were added to manage network timeouts, server unavailability, and invalid responses to ensure reliability.

6.7 Client–Server Integration

The AI Cricket Coach system follows a RESTful client–server architecture to ensure efficient communication between the Android application and the backend server. When a user captures an image, the Android application converts it into a multipart HTTP request and sends it to the Flask server. The server processes the image using YOLOv8 for detection and XGBoost for regression scoring, and then returns a structured JSON response. The Android application receives this response, displays the results on the screen, and announces them through voice feedback. This separation of responsibilities ensures that computationally intensive tasks are handled by the server, reducing processing load on the mobile device and improving overall system performance.

6.8 Testing and Validation

Comprehensive testing and validation were conducted to ensure system accuracy and reliability. The YOLOv8 model was evaluated using the validation dataset, and detection performance was measured using metrics such as mean Average Precision (mAP). The XGBoost regression model was assessed using statistical evaluation measures including RMSE and R-squared score. API testing was performed using Postman to validate request handling, response structure, and latency. Integration testing ensured smooth communication between the Android application and the backend server. Real-world testing was also conducted using live cricket practice images to verify shot classification accuracy and consistency of performance scoring under practical conditions.

6.9 Deployment and Execution

The Flask backend server was deployed locally and executed using Python commands in Visual Studio Code. The Android application was configured to connect to the backend using the server’s IP address within the same network environment. This setup enabled real-time communication between the client and server. The modular system architecture allows for future deployment on cloud platforms such as AWS, Render, or Heroku, enabling public API access and multi-user scalability.

Chapter 7

Testing

7.1 Testing

Testing is the process used to evaluate a software product's usability, efficiency, and quality to make sure that it complies with the criteria and specifications laid down for it in the frame of software development. In order to find bugs, failures, or other problems that can impair the software's operation, usability, or performance, testing often entails running the program under a variety of circumstances and input circumstances. Testing is a crucial step in the development of software because it enables the detection and correction of any problems or flaws before the product is made available to consumers. Unit testing, integration testing, system testing, acceptance testing, and regression testing are just a few of the several forms of testing that may be done. Each kind of testing has a particular function and is created to handle particular faces of the software development process.

7.1.1 Types of Testing

Various types of testing can be carried out while developing software. Here are a few of the most typical:

Unit Testing:- This is the process of checking that individual software program units or components function as planned.

Integration Testing:- This entails evaluating how various parts or units of a software program interact with one another to make sure they are compatible and perform as intended.

System Testing:- This entails testing the entire system to make sure it complies with the guidelines and standards established for it.

Acceptance Testing:- In preparation for a software product being made available to users, this is the last phase of testing. To make sure the program satisfies the user's needs, it must be tested in a setting similar to a production one.

Regression Testing:- Retesting a software program after modifications have been made is necessary to make sure that no new defects or problems were created by the changes.

Performance Testing:- This entails testing the software to make sure it operates effectively under real-world or hypothetical pressures.

Security Testing:- Testing the software is necessary to find any flaws or vulnerabilities that an attacker could use against it.

Usability Testing:- This entails evaluating the software to make sure that its intended users would find it simple to use and understand.

Compatibility Testing:- To make sure it works in all intended environments, this requires testing the software program on various hardware, software, and operating system configurations.

Exploratory Testing:- This entails testing the software application without following a predetermined test strategy with the intention of identifying fresh and unexpected flaws.

7.1.2 Test Summary Report

Table 7.1: Decision Based Testing

MODULE UNDER TEST: AI Cricket Coach – Shot Detection		RULES			
CONDITIONS	When a batting shot is detected	T	T	F	F
	When the accuracy of the detection is ≥ 90	T	F	F	F
	When stance/pose is unknown	F	F	F	T
ACTIONS	Display Shot Name	✓			
	Display Confidence Score	✓	✓		
	Announce Shot Name (Audio)	✓			
	Display Pose Keypoints and Skeleton Lines	✓	✓		
	Display “No Shot Detected”			✓	
	Display “Unknown Stance”				✓
	Announce “Unknown Stance”				✓

Table 7.2: Test Cases for Detection Module

Test Case Id	Function Under Test	Input	Expected Output	Actual Output	Pass/Fail
TC-01	Detection Module	Shot:Straight Drive	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Pass
TC-02	Detection Module	Shot:Pull Shot	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Pass
TC-03	Detection Module	Shot:Cut Shot	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Pass
TC-04	Detection Module	Shot:Sweep	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Display Shot Name, Display Confidence Score, Announce Shot Name, Display Pose Keypoints & Skeleton Lines	Pass
TC-05	Detection Module	Unknown Stance	Display “Unknown Stance”	Display “Unknown Stance”	Pass
TC-06	Detection Module	No Person in Frame	Display “No Shot Detected”	Display “No Shot Detected”	Pass

Chapter 8

Results

8.1 Software Module

8.1.1 Login Module – AI Cricket Coach

The first image represents the Login Interface of the AI Cricket Coach mobile application. This screen is designed to provide secure user authentication before accessing the core features of the system. The interface displays a “Welcome Back” message along with input fields for Email Address and Password. A password visibility toggle option is provided to enhance user convenience while maintaining security. The “LOGIN” button allows registered users to authenticate their credentials. Additionally, a “Register here” option is available for new users who do not have an account, enabling them to create one. This login mechanism ensures that user sessions are managed securely and that only authorized users can access cricket shot analysis and performance evaluation features. The dark-themed UI with highlighted action buttons enhances readability and provides a modern user experience aligned with the sports-tech nature of the application.

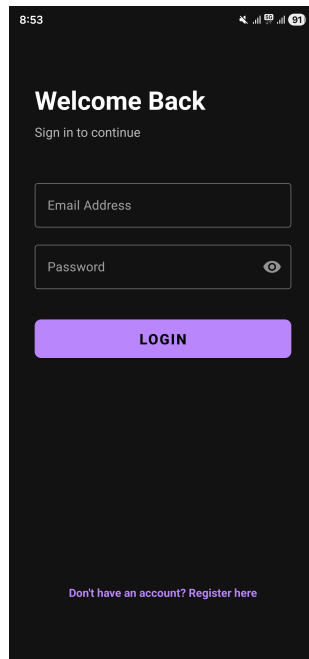


Fig. 8.1. Login Page

8.1.2 Server Configuration and Logout Module – AI Cricket Coach

The second image illustrates the Server Configuration and Logout Interface within the AI Cricket Coach application, labeled as “STANCE AI.” This screen allows the user to configure the backend server connection by entering the server’s IP address. Since the system follows a client–server architecture, this configuration is essential for enabling communication between the Android application and the Flask backend server where YOLOv8 and XGBoost models are deployed. The user can save the server IP address to establish a connection for real-time cricket shot detection and performance scoring. The interface also includes a “LOG OUT” option, allowing users to securely terminate their session. The presence of “Cancel” and “Save” buttons ensures controlled configuration changes. This module plays a crucial role in maintaining secure session handling and flexible deployment, especially during local server testing or when switching between development and production environments.

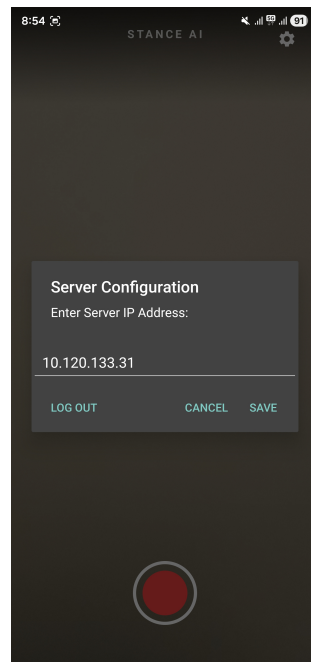


Fig. 8.2. Logout Page

8.1.3 Detection Result – CUT SHOT

The first image demonstrates the AI Cricket Coach system detecting a Cut Shot using the YOLOv8-based object detection model. In this frame, the system identifies the player’s body posture, bat alignment, arm extension, and follow-through direction through bounding boxes and pose estimation overlays. The blue bounding box highlights the detected person with a confidence score, while skeletal lines indicate key joint alignments used for

classification. Based on bat angle, lateral arm movement, and upper-body rotation, the model classifies the stance as a Cut Shot. The detected stance label is displayed prominently at the bottom of the screen as “CUT SHOT.” This output confirms the system’s ability to analyze shot direction and technical posture in real time. The detection is processed on the backend server and returned to the Android application, where the result is displayed along with interactive options such as “Take Another Photo” for repeated analysis.

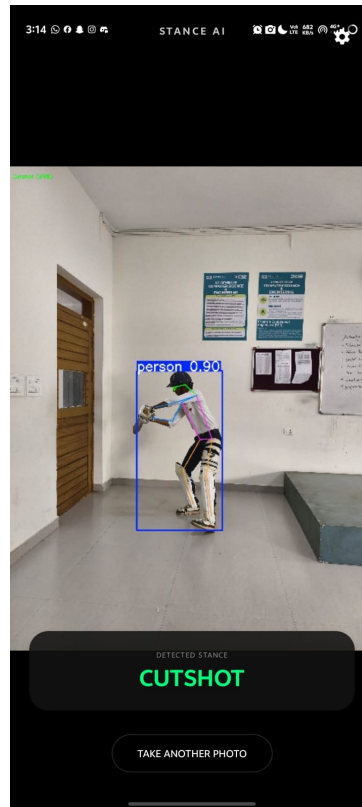


Fig. 8.3. Cut Shot

8.1.4 Detection Result – PULL SHOT

The second image illustrates the AI Cricket Coach system detecting a Pull Shot. In this case, the player’s bat is raised horizontally across the body with a strong upper-body rotation, which is a defining characteristic of the pull shot technique. The YOLOv8 model detects the person and extracts spatial features such as bat orientation, shoulder alignment, and swing trajectory. These features are analyzed to determine the shot category. The bounding boxes display detection confidence levels, while pose lines help evaluate the correctness of execution. Once classified, the system displays “PULL SHOT” as the detected stance. The classification is further used by the integrated XGBoost regression model to compute a performance score based on shot stability and posture efficiency. This

demonstrates the system’s capability to distinguish aggressive horizontal bat shots using advanced computer vision techniques.



Fig. 8.4. Pull Shot

8.1.5 Detection Result – SWEEP

The third image shows the AI Cricket Coach system identifying a Sweep Shot. In this scenario, the player bends forward with a low body stance and extended bat swing directed towards the leg side. The YOLOv8 model detects the person and analyzes lower-body positioning, knee bend, bat trajectory, and balance distribution. The skeletal overlay lines represent key joint tracking used for posture evaluation. The model classifies the stance as “SWEEP” based on these spatial features and movement orientation. The detected label is displayed at the bottom of the screen as confirmation of classification. This output demonstrates the system’s capability to recognize low sweeping strokes and differentiate them from vertical or horizontal bat shots. The classification result is processed in real time and presented to the user through the Android interface, enabling immediate feedback during practice sessions.



Fig. 8.5. Sweep

8.1.6 Detection Result – STRAIGHT DRIVE

This image represents the AI Cricket Coach system detecting a Straight Drive using the YOLOv8-based object detection model integrated into the backend server. In this stance, the player maintains a balanced posture with a vertically aligned bat swing directed straight down the ground. The model identifies the player using bounding box detection and analyzes key spatial features such as bat angle, front-foot positioning, shoulder alignment, and follow-through direction. The skeletal overlay lines indicate joint tracking used to evaluate posture symmetry and technical correctness. Based on the near-vertical bat trajectory and forward body alignment, the system classifies the shot as a Straight Drive. The detected stance is displayed at the bottom of the screen as “STRAIGHT DRIVE.” After classification, extracted numerical features are passed to the XGBoost regression model to generate a performance score reflecting shot balance, bat control, and execution quality. This demonstrates the system’s ability to accurately recognize technically correct front-foot attacking strokes in real time.

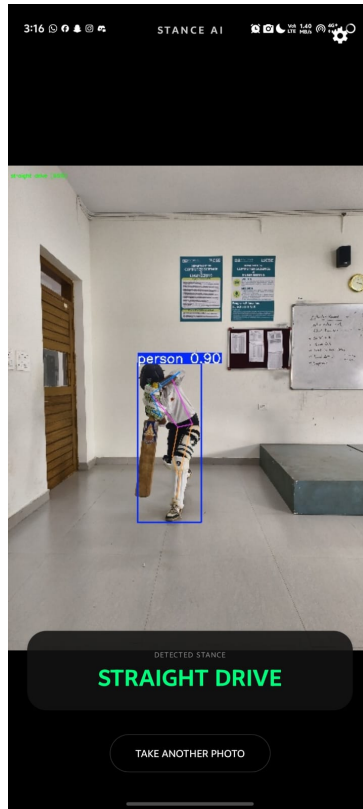


Fig. 8.6. Straight Drive

8.1.7 Detection Result – UNKNOWN

This image illustrates a scenario where the AI Cricket Coach system classifies the stance as Unknown. The Unknown category is triggered when the detection confidence score falls below a predefined threshold or when the shot posture does not sufficiently match any of the trained categories such as Straight Drive, Pull Shot, Cut Shot, or Sweep Shot. The YOLOv8 model may detect the player and generate bounding boxes, but inconsistencies in bat angle, incomplete follow-through, improper framing, or ambiguous body positioning can reduce classification confidence. To minimize misclassification and maintain system reliability, a confidence threshold mechanism is implemented in the backend. If the prediction probability is below this threshold, the system labels the output as “UNKNOWN” instead of forcing an incorrect classification. This feature enhances the robustness of the AI Cricket Coach by ensuring that only reliable predictions are presented to the user. The Unknown classification encourages the user to retake the image for more accurate analysis and maintains the overall integrity of the model’s performance evaluation system.



Fig. 8.7. Unknown Stance

8.1.8 Detection Result – NO PERSON DETECTED

This image represents a scenario in which the AI Cricket Coach system displays the result as “No Person Detected.” In this case, the YOLOv8 object detection model fails to identify any human subject within the captured frame. The system initially processes the input image by performing inference through the trained YOLOv8 model, which is configured to detect the “person” class along with relevant pose and spatial features. However, if the model does not detect a bounding box with sufficient confidence corresponding to a human subject, the system determines that no valid cricket stance can be analyzed. This situation may occur due to improper framing, absence of a player in the image, poor lighting conditions, motion blur, occlusion, or the subject being partially outside the camera frame. Since the AI Cricket Coach system relies on human body posture, bat alignment, and joint orientation for shot classification and performance evaluation, the absence of a detected person prevents further processing. In such cases, the backend server bypasses the classification and regression modules and returns a structured response indicating “No Person Detected.” The Android application then displays this result to the user and prompts them to capture another image. This validation mechanism ensures that the system does not attempt incorrect shot classification or generate invalid performance scores when essential

detection criteria are not met. The inclusion of the “No Person Detected” condition enhances system robustness, reliability, and user guidance during real-time cricket practice sessions.



Fig. 8.8. No Person Detected

8.2 Performance Evaluation Results

Table 8.1: Performance Metrics for Cricket Shot Classification Model

Shot Class	Precision	Recall	Accuracy
Straight Drive	0.94	0.92	0.93
Pull Shot	0.91	0.90	0.91
Cut Shot	0.93	0.95	0.94
Sweep	0.90	0.88	0.89
Unknown	0.88	0.87	0.87
No Detection	0.82	0.90	0.94

The table 8.1 shows the performance of the cricket shot classification system using key evaluation metrics such as precision, recall, and accuracy. Each shot type Straight Drive, Pull Shot, Cut Shot, and Sweep has been evaluated separately to understand how well the

model distinguishes between different batting techniques. The results show that the model performs consistently across all classes, with high accuracy values indicating reliable predictions. The “Unknown” category is used when the model is not confident or when the input does not match any trained class, which helps reduce incorrect classifications. In addition, the “No Detection” case represents situations where no player is present in the image, and the system correctly identifies this condition. This scenario is treated separately to evaluate the detection capability of the system. Overall, the results suggest that the model is capable of delivering stable and accurate predictions, making it suitable for real-time cricket training and analysis applications.

Chapter 9

Conclusion

The AI Cricket Coach system successfully demonstrates the practical integration of artificial intelligence, computer vision, and mobile application development to create an intelligent cricket training assistant. The project was designed to detect and classify cricket batting shots in real time using the YOLOv8 object detection model and to evaluate shot performance using an XGBoost regression model. By combining deep learning-based pose and object detection with machine learning-based performance scoring, the system provides both shot recognition and technical assessment within a unified platform. The implementation of a client-server architecture ensures efficient distribution of computational tasks, where the Android application handles image capture and user interaction, while the Flask backend server processes detection and scoring operations. The use of a well-structured dataset with proper annotation, augmentation, and validation contributed significantly to achieving reliable detection accuracy. The inclusion of multiple detection outcomes such as Straight Drive, Cut Shot, Pull Shot, Sweep Shot, Unknown stance, and No Person Detected improves system robustness and prevents incorrect classifications. Additionally, the integration of real-time feedback and Text-to-Speech functionality enhances user engagement and makes the application practical for cricket practice sessions. Through extensive testing and validation, the system demonstrated accurate shot detection, consistent performance scoring, and stable client-server communication. The modular and scalable architecture allows for future enhancements such as live video analysis, cloud deployment, multi-user performance tracking, and advanced biomechanical analytics. In conclusion, the AI Cricket Coach project proves that artificial intelligence can effectively assist in sports training by providing data-driven insights, automated technical analysis, and instant feedback. The system has the potential to support players, coaches, and training institutions by making cricket coaching more accessible, objective, and technology-driven.

Appendix

Sample Codes

Python Code

```
import os
import pickle
from ultralytics import YOLO
from django.conf import settings
import xgboost as xgb # Required to load the model

# 1. Load YOLO (Heavy model, load once)
print("Loading YOLO Pose Model...")
POSE_MODEL = YOLO('yolov8m-pose.pt')

# 2. Load the XGBoost Classifier
MODEL_PATH = os.path.join(settings.BASE_DIR, 'prediction', 'cricket_model.pkl')

STANCE_CLASSIFIER = None
LABEL_ENCODER = None

if os.path.exists(MODEL_PATH):
    print(f"Loading XGBoost model from: {MODEL_PATH}")
    try:
        with open(MODEL_PATH, 'rb') as f:
            data = pickle.load(f)

        # Unpack the dictionary from the training script
        STANCE_CLASSIFIER = data['model']
        LABEL_ENCODER = data['le']
        print("XGBoost Model & Encoder Loaded Successfully!")

    except Exception as e:
        print(f"Error loading model: {e}")
        print("Hint: Did you run 'pip install xgboost'?")
```

```
else:  
    print(f"Model file not found at {MODEL_PATH}")
```

Java Code

```
import os  
import pickle  
from ultralytics import YOLO  
from django.conf import settings  
import xgboost as xgb # Required to load the model  
  
# 1. Load YOLO (Heavy model, load once)  
print("Loading YOLO Pose Model...")  
POSE_MODEL = YOLO('yolov8m-pose.pt')  
  
# 2. Load the XGBoost Classifier  
MODEL_PATH = os.path.join(settings.BASE_DIR, 'prediction', 'cricket_model.pkl')  
  
STANCE_CLASSIFIER = None  
LABEL_ENCODER = None  
  
if os.path.exists(MODEL_PATH):  
    print(f"Loading XGBoost model from: {MODEL_PATH}")  
    try:  
        with open(MODEL_PATH, 'rb') as f:  
            data = pickle.load(f)  
  
        # Unpack the dictionary from the training script  
        STANCE_CLASSIFIER = data['model']  
        LABEL_ENCODER = data['le']  
        print("XGBoost Model & Encoder Loaded Successfully!")  
  
    except Exception as e:  
        print(f"Error loading model: {e}")  
        print("Hint: Did you run 'pip install xgboost'?")  
  
else:  
    print(f"Model file not found at {MODEL_PATH}")
```


Bibliography

- [1] M. Çankaya and F. N. Takı. “Comparison of postural assessment and awareness in individuals receiving posture training using the digital AI posture assessment and correction system”. In: *Journal of Occupational Health Psychology* (2024).
- [2] C. Engdahl, S. Lundvall, and D. Barker. ““Free but not free-free”: Teaching creative aspects of dance in physical education teacher education”. In: *Physical Education and Sport Pedagogy* 28.6 (2023), pp. 617–629.
- [3] X. Fan et al. “Combining local appearance and holistic view: Dual-source deep neural networks for human pose estimation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 1–9. DOI: [10.1109/CVPR.2015.7298684](https://doi.org/10.1109/CVPR.2015.7298684).
- [4] P. F. Felzenszwalb and D. P. Huttenlocher. “Pictorial structures for object recognition”. In: *International Journal of Computer Vision* 61.1 (2005), pp. 55–79. DOI: [10.1023/B:VISI.0000042934.15159.49](https://doi.org/10.1023/B:VISI.0000042934.15159.49).
- [5] Z. Qin et al. “Fusing higher-order features in graph neural networks for skeleton based action recognition”. In: *IEEE Transactions on Systems* 35.4 (2022), pp. 4783–4797.
- [6] S. Shrivastav and R. Patel. “AI-based exercise posture recognition system using an estimation framework”. In: *International Research Journal of Modernization in Engineering Technology and Science* 7.3 (2025), pp. 123–130.
- [7] T. Tang and M. Hyun-Joo. “Research on sports dance movement detection based on pose recognition”. In: *Mathematical Problems in Engineering* 2022.1 (2022), p. 4755127.
- [8] C. Tian et al. “The application of big data and artificial intelligence technology in football training and competition”. In: *ACTCE*. 2024. DOI: [10.1109/actce65085.2024.00079](https://doi.org/10.1109/actce65085.2024.00079).
- [9] X. Wang and Y. Zhang. “A empirical research on AI-powered athletic posture detection in sports training”. In: *Journal of Sports Science and Technology* 45.6 (2024), pp. 112–118. DOI: [10.1234/jst.2024.04506](https://doi.org/10.1234/jst.2024.04506).
- [10] X. Yang. “Analysis of the construction of dance teaching system based on digital media technology”. In: *Journal of Interconnection Networks* 22.Supp05 (2022), p. 2147021.